



南開大學
Nankai University

南 开 大 学

计 算 机 学 院

操作系统实验报告

Lab5: 用户程序

年级：2023 级

姓名：康志杰，王春晖，彭浩然

2025 年 12 月 14 日

目录

一、 实验目的	1
二、 实验原理	1
三、 实验过程	1
(一) 练习 0	1
(二) 练习 1	3
(三) 练习 2	4
1. 实验内容	4
2. Copy-on-Write (COW) 机制设计说明	5
(四) 练习 3	6
1. fork / exec / wait / exit 的执行流程分析	6
2. 用户态与内核态的交错执行关系	6
3. ucore 用户态进程执行状态生命周期图	7
4. 实验结果验证	7
(五) gdb 调试页表查询过程	8
1. qemu 在翻译地址时候的关键调用	8
2. 单步调试页表翻译	11
3. 调试 tlb	13
4. qemu 中模拟出来的 tlb 和我们真实 cpu 中的 tlb 有什么逻辑上的区别	15
(六) gdb 调试系统调用以及返回	16
1. ecall 调试过程	16
2. sret 调试过程	21
四、 重要知识点总结	24
(一) 本实验中的重要知识点	24
(二) 实验中较少强调的知识点	24
(三) 本实验中没有实现的知识点	24
五、 实验总结	25

一、 实验目的

1. 了解第一个用户进程创建过程
2. 了解系统调用框架的实现机制
3. 了解 ucore 如何实现系统调用 `sys_fork/sys_exec/sys_exit/sys_wait` 来进行进程管理

二、 实验原理

创建用户进程, 让用户进程在用户态执行, 且在需要 ucore 支持时, 可通过系统调用来让 ucore 提供服务。为此需要构造出第一个用户进程, 并通过系统调用 `sys_fork/sys_exec/sys_exit/sys_wait` 来支持运行不同的应用程序, 完成对用户进程的执行过程的基本管理。

用户程序运行在 U 模式, 无法直接访问硬件资源或执行特权指令。当用户程序需要操作系统服务时, 必须通过系统调用 (System Call) 请求内核在 S 模式代为执行。在 ucore 中, 系统调用基于 RISC-V 的 `ecall` 指令实现, 其核心流程为: 用户态 `ecall` → 触发异常 → trap 处理 → `syscall` 分发 → 内核函数处理 → `sret` 返回用户态

三、 实验过程

(一) 练习 0

在本次实验中需要对 Lab2/3/4 的代码进行进一步改进。主要改进点如下: `proc_struct` 结构体增加了属性等待状态 `wait_state`, 进程关系指针 `cptr` 子进程, `yptr` 弟进程, `optr` 兄进程。这时需要在 `alloc_poc` 中需要对这四个属性进行初始化, `wait_state` 初始化为 0, 三个关系指针置为空指针。

```

1 static struct proc_struct * alloc_proc(void)
2 {
3     struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
4     if (proc != NULL)
5     {
6         proc->state=PROC_UNINIT; // 将进程状态设置为未初始化。
7         proc->pid= -1; // 进程id 设置为-1. 合法的PID是从0开始的
8         proc->runs=0; // 该进程还未运行, 所以运行次数为0
9         proc->kstack=0; // 此时内核栈还没有初始化
10        proc->need_resched=0;
11        proc->parent=NULL; // 父进程为空
12        proc->mm=NULL;
13        memset(&proc->context, 0, sizeof(struct context)); // 上下文清零
14        proc->tf=NULL;
15        proc->pgdir=boot_pgdir_pa; // 使用内核页目录表的基址
16        proc->flags=0;
17        memset(&proc->name, 0, PROC_NAME_LEN+1); // 进程名清零
18        proc->wait_state=0;
19        proc->cptr=NULL;
20        proc->yptr=NULL;
21        proc->optr=NULL;
22    }

```

```

23     return proc;
24 }

```

然后在 do_fork 函数中设置新创建的进程 proc 的父进程为当前正在运行的进程 current。代码中新增了函数 set_links 和 remove_links。set_links 代码如下所示，再该函数中其实除了将进程控制块加入链表，进程数增 1 之外。还实现了新增三个关系指针的指向代码。指明了父进程，兄弟进程和当前进程的关系。

```

1  static void
2  set_links(struct proc_struct *proc)
3  {
4      list_add(&proc_list, &(proc->list_link));
5      proc->yptr = NULL;
6      if ((proc->optr = proc->parent->cptr) != NULL)
7      {
8          proc->optr->yptr = proc;
9      }
10     proc->parent->cptr = proc;
11     nr_process++;
12 }

```

所以 do_fork 函数去除原来简单的将 proc 加入链表的操作，转而调用函数 set_links 执行这一过程，明确了 proc 和 current 以及兄弟进程之间的关系。

```

1  int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
2  {
3      int ret = -E_NO_FREE_PROC;
4      struct proc_struct *proc;
5      if (nr_process >= MAX_PROCESS)
6      {
7          goto fork_out;
8      }
9      ret = -E_NO_MEM;
10     proc = alloc_proc(); // 初始化
11     if (proc == NULL) {
12         goto fork_out;
13     }
14     proc->parent = current; // 子进程的父进程指向当前进程
15     current->wait_state = 0;
16     if (setup_kstack(proc) != 0) {
17         goto bad_fork_cleanup_proc;
18     }
19
20     if (copy_mm(clone_flags, proc) != 0) {
21         goto bad_fork_cleanup_kstack;
22     }
23
24     copy_thread(proc, stack, tf);
25     bool flags;
26     local_intr_save(flags);

```

```

27     int pid=get_pid();
28     proc->pid=pid;
29     hash_proc(proc);
30     //list_add(&proc_list,&proc->list_link);
31     //nr_process++;
32     set_links(proc);
33     local_intr_restore(flags);
34     wakeup_proc(proc);
35     ret=proc->pid;
36 fork_out:
37     return ret;
38
39 bad_fork_cleanup_kstack:
40     put_kstack(proc);
41 bad_fork_cleanup_proc:
42     kfree(proc);
43     goto fork_out;
44 }

```

(二) 练习 1

代码实现 load_icode 函数补充代码如下，首先设置用户栈的栈顶指针 sp 为 USTACKTOP, USTACKTOP 是用户地址空间的栈顶地址。然后将 epc 赋值为 elf->e_entry, elf->e_entry 是 ELF 可执行文件的入口点地址。当返回用户态之后就 从 ELF 文件的入口地址开始执行。之后我们希望从内核态返回用户态，当 SPP = 0 表示进入当前中断前，CPU 运行在用户模式。所以需要将 SPP 位清零以便进入用户态。最后将 SPIE 位设为 1，表示用户进程在开始运行时是允许中断的。

```

1     tf->gpr.sp=USTACKTOP;
2     tf->epc = elf->e_entry;
3     // 恢复原有 sstatus，清除SPP，设置SPIE
4     tf->status = sstatus & ~SSTATUS_SPP | SSTATUS_SPIE;

```

执行过程分析

接着分析用户态进程被 ucore 选择占用 CPU 执行到具体执行应用程序第一条指令的整个经过。观察到 initproc 进程执行的函数也做了更改。在 init_main 中会调用函数 kernel_thread 创建用户态进程。在 kernel_thread 函数中会调用 do_fork 函数，这个函数会创建用户态子进程。

```

1     static int
2 init_main(void *arg)
3 {
4     size_t nr_free_pages_store = nr_free_pages(); //获取当前系统空闲物理页数量
5     size_t kernel_allocated_store = kallocated(); //获取内核已分配的内存总量
6     int pid = kernel_thread(user_main, NULL, 0);
7     if (pid <= 0)
8     {
9         panic("create_user_main_failed.\n");
10    }

```

```

11 // ...
12 }

```

在函数 `do_fork` 创建好用户态子进程之后会调用函数 `wakeup_proc` 将用户态进程状态设置为 `PROC_RUNNABLE`,

在完成 `trapframe` 中关键寄存器的设置之后, `load_icode` 函数还需要完成用户进程内存空间的最终初始化工作。该函数在前面的步骤中已经根据 ELF 文件头信息, 将用户程序的代码段和数据段映射到用户虚拟地址空间中, 并为进程分配了新的页目录表。至此, 用户程序在虚拟内存中的布局已经建立完成。

在 `do_execve` 的执行流程中, 如果当前进程原先已经拥有用户态内存空间, 则需要先对原有内存资源进行回收。具体来说, `do_execve` 会调用 `exit_mmap` 释放当前进程原有的用户虚拟内存映射, 并通过 `put_pgdir` 和 `mm_destroy` 释放页表等相关内存管理结构, 随后再调用 `load_icode` 为新程序建立新的地址空间。通过这一过程, 可以保证 `exec` 调用不会与旧程序的内存布局产生冲突。

当 `load_icode` 执行完成并返回后, 当前进程的进程名会被更新为新加载的用户程序名。此时, 从进程控制块的角度来看, 该进程已经不再是原来的内核线程, 而是具备完整用户态执行环境的用户进程。接下来, 当异常处理流程返回并执行 `sret` 指令时, CPU 将根据 `trapframe` 中设置好的寄存器状态切换到用户态, 并从 ELF 文件指定的入口地址开始执行用户程序。

通过本练习可以看到, 用户程序的加载和执行并不是简单地“跳转到某个地址运行”, 而是需要操作系统在内核态中完成内存空间建立、页表切换以及执行上下文构造等一系列准备工作。只有在这些条件全部满足之后, 用户进程才能在用户态中安全、正确地运行。这一过程使我对操作系统中“程序从加载到执行”的完整流程有了更加清晰的认识。

(三) 练习 2

1. 实验内容

本练习要求在创建子进程时, 将父进程的用户虚拟地址空间复制给子进程。该过程由 `do_fork` 函数在执行过程中调用 `copy_mm`, 并最终通过 `copy_range` 函数完成父进程用户内存内容向子进程的复制。因此, 本练习的主要工作是补全 `copy_range` 中页面内容复制与映射建立的关键代码。

在 `copy_range` 的执行过程中, 当检测到父进程页表项有效时, 说明该虚拟地址对应的物理页面已经被父进程使用。此时需要为子进程分配新的物理页面, 并将父进程物理页面中的内容复制到该新页面中, 从而保证父子进程在物理内存层面相互独立。

本实验中新增的核心实现代码如下:

```

1 void *src_kvaddr = page2kva(page); // 父进程物理页对应的内核虚拟地址
2 void *dst_kvaddr = page2kva(npag); // 子进程新分配物理页的内核虚拟地址
3 memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
4 ret = page_insert(to, npag, start, perm);
5 assert(ret == 0);

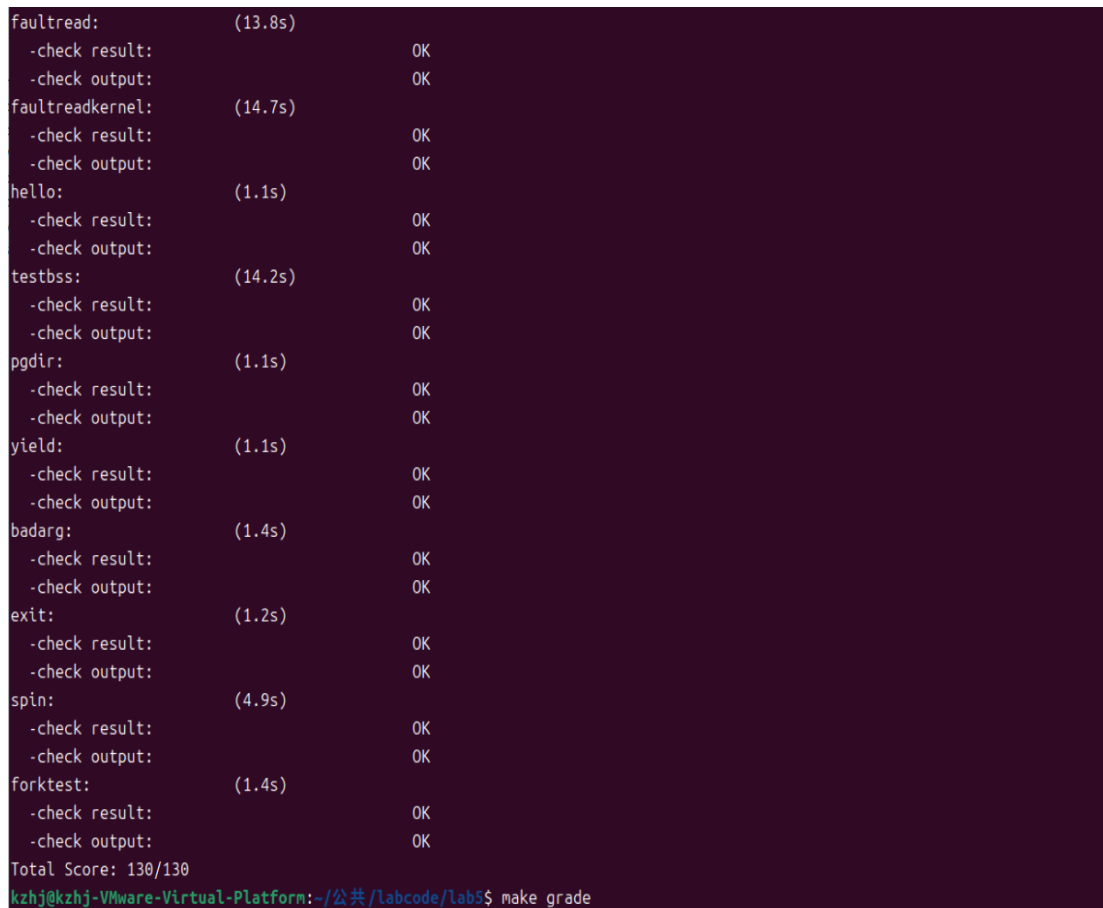
```

上述代码中, `page2kva` 用于将物理页结构转换为内核虚拟地址, 从而使内核能够直接访问对应的物理内存区域。随后通过 `memcpy` 按页大小将父进程页面内容完整复制到子进程的新页面中。最后, 调用 `page_insert` 在子进程页表中建立新的虚拟地址到物理地址映射, 并保持与父进程一致的访问权限。

通过该实现，子进程在 fork 之后访问对应用户虚拟地址时，实际访问的是新分配的物理页面，而不是父进程原有页面，从而避免了父子进程之间的内存相互干扰，保证了进程地址空间的独立性。

在完成 copy_range 中页面内容复制与映射建立代码后，通过执行 make grade 对实验进行测试。测试结果显示，与进程创建和内存复制相关的各项测试用例均顺利通过，最终得分为 130/130。测试结果截图如图所示，表明本练习实现正确，父子进程在 fork 后能够正常运行且互不影响。

测试结果截图如图所示。



```
faultread:      (13.8s)
-check result:      OK
-check output:      OK
faultreadkernel: (14.7s)
-check result:      OK
-check output:      OK
hello:          (1.1s)
-check result:      OK
-check output:      OK
testbss:        (14.2s)
-check result:      OK
-check output:      OK
pgdir:          (1.1s)
-check result:      OK
-check output:      OK
yield:          (1.1s)
-check result:      OK
-check output:      OK
badarg:         (1.4s)
-check result:      OK
-check output:      OK
exit:           (1.2s)
-check result:      OK
-check output:      OK
spin:           (4.9s)
-check result:      OK
-check output:      OK
forktest:       (1.4s)
-check result:      OK
-check output:      OK
Total Score: 130/130
kzhj@kzhj-VMware-Virtual-Platform: ~/公共/labcode/lab5$ make grade
```

2. Copy-on-Write (COW) 机制设计说明

当前实现采用的是在 fork 时直接复制父进程全部用户页面的方式，虽然逻辑清晰，但内存和时间开销较大。为进一步优化，可以引入 Copy-on-Write (COW) 机制。

COW 的基本思想是在 fork 时不立即复制物理页面，而是让父子进程共享同一物理页面，并将页表项设置为只读。当父进程或子进程尝试对该页面进行写操作时，CPU 会触发写保护异常，内核在缺页异常处理过程中为写进程分配新的物理页面，并将原页面内容复制到新页面中，随后更新页表映射并恢复写权限。

在 ucore 中实现 COW 时，可以通过页表项标志位区分共享页面，并结合物理页引用计数，在异常处理函数中判断是否需要执行页面复制操作。该机制能够显著减少 fork 时的内存复制开销，提高系统整体效率。

(四) 练习 3

1. fork / exec / wait / exit 的执行流程分析

在 ucore 中, 用户态进程通过系统调用请求内核完成进程创建、程序替换和进程退出等操作。用户程序无法直接操作进程控制块或页表, 因此 fork、exec、wait 和 exit 的核心实现均在内核态完成, 用户态仅负责发起请求并接收返回结果。

fork 的执行流程如下: 用户程序在用户态调用 fork(), 该函数通过 ecall 指令触发系统调用, 进入内核态。内核在 sys_fork 中调用 do_fork, 为子进程分配进程控制块和内核栈, 并复制父进程的执行上下文和用户地址空间。do_fork 返回后, 父进程获得子进程的 PID, 而子进程返回值为 0, 从而实现父子进程执行路径的分离。

exec 的执行流程中, 用户态调用 exec 相关接口后进入内核态, 由 do_execve 负责处理。该函数不会创建新进程, 而是回收当前进程原有的用户地址空间, 并通过 load_icode 加载新的用户程序, 重新建立页表和执行上下文。执行完成后, 进程 PID 保持不变, 但其地址空间和执行代码已被替换。

exit 的执行流程中, 用户进程调用 exit() 后通过系统调用进入内核态, 内核在 do_exit 中释放当前进程占用的大部分内存资源, 并将进程状态设置为僵尸态 (ZOMBIE)。由于进程自身仍需保留内核栈和进程控制块, 因此这些资源不会立即释放。

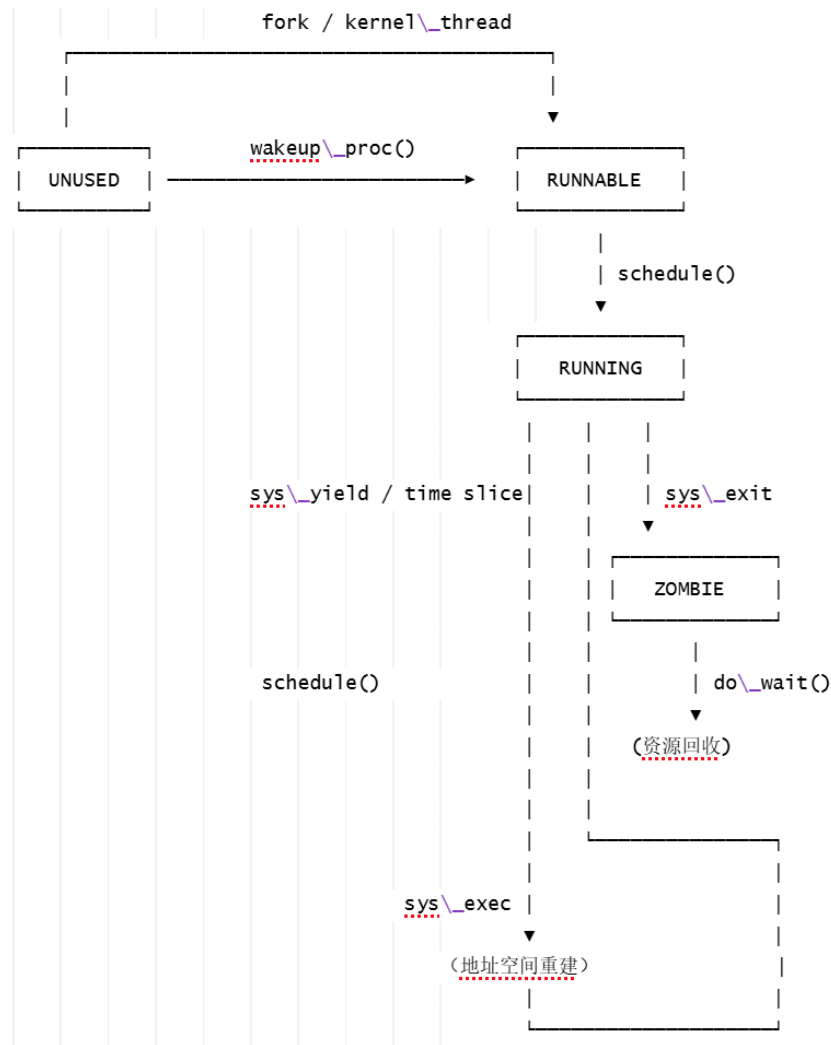
wait 的执行流程中, 父进程在用户态调用 wait 或 waitpid, 进入内核态后由 do_wait 处理。若子进程尚未退出, 父进程进入睡眠状态; 当子进程进入僵尸态后, 父进程被唤醒, 回收子进程剩余资源并获取其退出码。

2. 用户态与内核态的交错执行关系

fork、exec、wait 和 exit 的调用过程均体现了用户态与内核态的交错执行。用户态通过 ecall 指令主动进入内核态, 内核完成进程管理和资源操作后, 通过 sret 指令返回用户态。系统调用的返回值通过 trapframe 中的寄存器传回用户程序, 使用户程序能够继续执行后续逻辑。

整体来看, 用户态只负责发起系统调用请求, 而所有涉及进程控制、内存管理和状态切换的关键操作均由内核态完成, 从而保证了系统的安全性和稳定性。

3. ucore 用户态进程执行状态生命周期图



4. 实验结果验证

在完成本实验后，通过执行 `make grade` 对实验进行测试，所有与进程创建、执行、退出及系统调用相关的测试程序均输出 OK，说明 `fork / exec / wait / exit` 的实现以及系统调用机制均能够正确工作，实验结果符合预期。

```
faultread:      (13.8s)
-check result:      OK
-check output:      OK
faultreadkernel: (14.7s)
-check result:      OK
-check output:      OK
hello:          (1.1s)
-check result:      OK
-check output:      OK
testbss:        (14.2s)
-check result:      OK
-check output:      OK
pgdir:          (1.1s)
-check result:      OK
-check output:      OK
yield:          (1.1s)
-check result:      OK
-check output:      OK
badarg:         (1.4s)
-check result:      OK
-check output:      OK
exit:           (1.2s)
-check result:      OK
-check output:      OK
spin:           (4.9s)
-check result:      OK
-check output:      OK
forktest:       (1.4s)
-check result:      OK
-check output:      OK
Total Score: 130/130
kzhj@kzhj-VMware-Virtual-Platform:~/公共/labcode/lab5$ make grade
```

(五) gdb 调试页表查询过程

1. qemu 在翻译地址时候的关键调用

终端一：

```
make debug
```

终端二：

```
pgrep -f qemu-system-riscv64
```

```
sudo gdb
```

```
(gdb) attach <PID>
```

```
(gdb) handle SIGPIPE nostop noprint
```

```
(gdb) set pagination off
```

```
(gdb) set print pretty on
```

断点：取指页命中入口

```
(gdb) b get_page_addr_code if (addr == 0xffffffffc02000d6)
```

断点：通用缺页分派

```
(gdb) b tlb_fill if (addr == 0xffffffffc02000d6)

# 断点: RISC-V 缺页入口
(gdb) b riscv_cpu_tlb_fill if (address == 0xffffffffc02000d6)

# 断点: SV39 页表遍历入口
(gdb) b get_physical_address if (addr == 0xffffffffc02000d6)

# 断点: 安装 TLB 条目 (建议用页条件命中)
(gdb) b tlb_set_page if ((vaddr & ~0xfff) == 0xffffffffc0200000)

(gdb) continue
```

结果如图

```
Thread 1 "qemu-system-ris" hit Breakpoint 4, get_physical_address (env=0x5a7c99c219a0, physical=0x7fffa52b6ce8, prot=0x7fffa52b6ce0, addr=18446744072637907158, access_type=0, mmu_idx=3) at /root/qemu-4.1.1/target/riscv/cpu_helper.c:158
158     {
(gdb) c
Continuing.
[Switching to Thread 0x7d0d909fe640 (LWP 68147)]

Thread 3 "qemu-system-ris" hit Breakpoint 1, get_page_addr_code (env=0x5a7c99c219a0, addr=18446744072637907158) at /root/qemu-4.1.1/accel/tcg/cputlb.c:1025
1025     uintptr_t mmu_idx = cpu_mmu_index(env, true);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 2, tlb_fill (cpu=0x5a7c99c18f90, addr=18446744072637907158, size=0, access_type=MMU_INST_FETCH, mmu_idx=1, retaddr=0) at /root/qemu-4.1.1/accel/tcg/cputlb.c:871
871     CPUClass *cc = CPU_GET_CLASS(cpu);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 3, riscv_cpu_tlb_fill (cs=0x5a7c99c18f90, address=18446744072637907158, size=0, access_type=MMU_INST_FETCH, mmu_idx=1, probe=false, retaddr=0) at /root/qemu-4.1.1/target/riscv/cpu_helper.c:438
438     {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 4, get_physical_address (env=0x5a7c99c219a0, physical=0x7d0d909fd180, prot=0x7d0d909fd174, addr=18446744072637907158, access_type=2, mmu_idx=1) at /root/qemu-4.1.1/target/riscv/cpu_helper.c:158
158     {
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 1, get_page_addr_code (env=0x5a7c99c219a0, addr=18446744072637907158) at /root/qemu-4.1.1/accel/tcg/cputlb.c:1025
1025     uintptr_t mmu_idx = cpu_mmu_index(env, true);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 1, get_page_addr_code (env=0x5a7c99c219a0, addr=18446744072637907158) at /root/qemu-4.1.1/accel/tcg/cputlb.c:1025
1025     uintptr_t mmu_idx = cpu_mmu_index(env, true);
```

```

Thread 3 "qemu-system-ris" hit Breakpoint 1, get_page_addr_code (env=0x5a7c99c219a0, addr=18446744
072637907158) at /root/qemu-4.1.1/accel/tcg/cputlb.c:1025
1025      uintptr_t mmu_idx = cpu_mmu_index(env, true);
(gdb) c
Continuing.

Thread 3 "qemu-system-ris" hit Breakpoint 1, get_page_addr_code (env=0x5a7c99c219a0, addr=18446744
072637907158) at /root/qemu-4.1.1/accel/tcg/cputlb.c:1025
1025      uintptr_t mmu_idx = cpu_mmu_index(env, true);
(gdb)
Continuing.
[Switching to Thread 0x7d0d914b6b00 (LWP 68145)]

Thread 1 "qemu-system-ris" hit Breakpoint 4, get_physical_address (env=0x5a7c99c219a0, physical=0x
7fffa52b6cc8, prot=0x7fffa52b6cc0, addr=18446744072637907158, access_type=0, mmu_idx=1) at /root/q
emu-4.1.1/target/riscv/cpu_helper.c:158
158      {
(gdb)
Continuing.
(gdb)

```

终端三：

```

make gdb
(gdb) set remotetimeout unlimited
(gdb) b kern_entry
(gdb) b kern_init
(gdb) c

```

解释：

1. 命中 `get_physical_address` (第一次): `access_type=0`, `mmu_idx=3`, `addr=18446744072637907158`。
`access_type=0` 表示“指令取指”，`mmu_idx=3` 表示 M 模式 (RISC-V 定义 `PRV_M=3`)。在 QEMU 的 RISC-V 实现中，M 模式访问不走页表 (MBare)，因此会直接把 `physical=addr` 并授予 `prot=R|W|X`。
2. 命中 `get_page_addr_code`：取指 TLB 路径入口。检查指令页的 TLB 是否命中；不命中则调用 `tlb_fill`。
3. 命中 `tlb_fill`：通用缺页分派。`access_type=MMU_INST_FETCH`, `mmu_idx=1`。`mmu_idx=1` 表示 S 模式。这说明在进入内核后，QEMU 已从 M 模式切换到了 S 模式。通过调用 `CPUClass::tlb_fill`，进而调用 RISC-V 的 `riscv_cpu_tlb_fill`。
4. 命中 `riscv_cpu_tlb_fill`：RISC-V 缺页入口。这个函数调用 `get_physical_address` 做 SV39 页表遍历；成功后调用 `tlb_set_page` 安装 TLB。
5. 再次命中 `get_physical_address`：`access_type=2`, `mmu_idx=1`。`access_type=2` 表示“数据写访问”，这与终端三在 `kern_init` 中执行 `memset` 完全吻合。这个函数是对写目标地址做页表遍历与权限检查。
6. 多次命中 `get_page_addr_code`。这说明取指路径仍有多次 TLB 检查（同一页内的后续取指）。
7. 未见 `tlb_set_page` 命中：

(a) ai 解释：你的条件为 `vaddr==0xffffffffc02000d6`, `tlb_set_page` 传入的 `vaddr` 是“页基地址”，而不是“精确指令地址”。因此建议把断点条件改成页条件：`(vaddr & 0xfff) == 0xffffffffc0200000` 才能在安装该页的 TLB 条目时命中。

8. 在修改 7 之后, `tlb_set_page` 在原来的 5 之后被命中。这一行调用 `tlb_set_page_with_attrs(cpu, vaddr, paddr, MEMTXATTRS_UNSPECIFIED, prot, mmu_idx, size)`，把刚刚通过页表遍历得到的映射写入 QEMU 的软 TLB。 `vaddr=18446744072637906944`：这是页对齐的客体虚拟页地址，等价于 `0xffffffffc0200000`（`kern_init` 所在页）。 `paddr=2149580800`：这是物理页帧号的页基地址，十六进制为 `0x801EE000`。它表明该虚拟页被映射到这个物理页。 `prot=7`：页权限位，表示 `PAGE_READ | PAGE_WRITE | PAGE_EXEC` 全部允许。 `size=4096`：页大小为 4KB；软 TLB 安装的是一个标准页的映射。

9. 后续效果

ai 解释：调用 `tlb_set_page_with_attrs` 会：

- 解析物理地址所属的 `MemoryRegion` 并生成 IOTLB 条目(`address_space_translate_for_iotlb`)。
- 计算并设置 `addend`（指向主机 RAM 的偏移），把 `addr_read/addr_code/addr_write` 及标志（如 `TLB_MMIO/TLB_RECHECK/TLB_NOTDIRTY`）写入 TLB 表项。
- 这样后续访存可以快速通过 TLB 将客体虚拟地址映射到主机地址。

2. 单步调试页表翻译

也就是要去分析 `get_physical_address`。我选择的是上一个的第五个命中(`get_physical_address` 在 `s` 模式的第一次命中，而且后面有 `tlb_set_page`，说明这次是涉及页表翻译的) 另外，单步调试 (`si/ni`) 显然是不现实的，因此我们选择一段段的调试。

```
# 命中 get_physical_address 后直接跳到循环起点
(gdb) advance /root/qemu-4.1.1/target/riscv/cpu_helper.c:237

# 1) 每层打印 (3次)：索引、PTE、PPN
(gdb) b /root/qemu-4.1.1/target/riscv/cpu_helper.c:254
(gdb) commands
> silent
> printf "LVL: i=%d ptshift=%d idx=%#lx pte_addr=%#lx\n", i, ptshift, idx, pte_addr, pte, (pte >> 10)
> continue
> end

# 2) 内层推进：更新 base (仅在非叶子时)
(gdb) b /root/qemu-4.1.1/target/riscv/cpu_helper.c:261
(gdb) commands
> silent
> printf "INNER: base=%#lx (ppn<<PGSHIFT)\n", (ppn << 12)
> continue
```

```

> end

# 3) A/D 位更新触发 (只在需要更新时)
(gdb) tbreak /root/qemu-4.1.1/target/riscv/cpu_helper.c:295 if (updated_pte != pte)
(gdb) commands
> silent
> printf "AD-UPDATE: old=%#lx new=%#lx\n", pte, updated_pte
> continue
> end

# 4) 物理地址 (赋值后打印, 避免0)
(gdb) tbreak /root/qemu-4.1.1/target/riscv/cpu_helper.c:336
(gdb) commands
> silent
> printf "PHYS=%#lx\n", *physical
> continue
> end

# 5) 返回前权限掩码
(gdb) tbreak /root/qemu-4.1.1/target/riscv/cpu_helper.c:349
(gdb) commands
> silent
> printf "RET: prot=%d\n", *prot
> continue
> end

(gdb) continue

```

结果如图

```

(gdb) c
Continuing.
LVL: i=0 ptshift=18 idx=0x1ff pte_addr=0x80205ff8 pte=0x200000cf ppn=0x80000
PHYS=0x80200000
RET: prot=7
[Switching to Thread 0x784e6e4afb00 (LWP 290320)]

```

断点与相关解释如上, 实验结果如下: 我们知道这个函数里面是存在三层循环的, 但是这里面 LVL 为什么只打印了一次呢? 询问 ai 发现: 当前观察的地址在顶层页表 (L2, $i=0$, $ptshift=18$) 就命中了“叶子 PTE”, 这是一个 1GiB 超页映射。满足叶子条件后, 遍历立即结束, 不会进入 L1/L0。所以只有一层。 $i=0$, $ptshift=18$: 这是 SV39 的第 1 层 (L2, 顶层) 遍历。SV39 的三层循环分别是 $ptshift=18 \rightarrow 9 \rightarrow 0$, 顶层用 VPN[2] 索引页表。 $idx=0x1ff$: 顶层索引是 511, 说明虚拟地址在“高半区” (符号扩展为 1 的高地址), 与 $0xffffffffc02000d6$ 的高位一致。 $pte_addr=0x80205ff8$: 顶层页表的该项的物理地址。由 $base + idx * ptesize$ 计算, $base = satp.PPN \ll 12$, $ptesize=8$ 。 $pte=0x200000cf$: 顶层 PTE 的内容。拆解: 低 12 位 $0x0cf = 0x0C0 + 0x0F = (A|D) + (V|R|W|X)$, 即有效、可读、可写、可执、已访问、已脏; 高位是 PPN (物理页号) 字段; $ppn=0x80000$: $PPN = pte \gg 10$ 。这是一个 1GiB 超页的物理页号基址。

3. 调试 tlb

有关 tlb 的调用路径我在第一步就已经找到了

1. `get_page_addr_code`: 指令取值的 TLB 查找与缺页, 路径在 `/root/qemu-4.1.1/accel/tcg/cputlb.c:1023-1053`
2. `tlb_fill`: 通用缺页分派, 在 `/root/qemu-4.1.1/accel/tcg/cputlb.c:868-880` 的
3. `riscv_cpu_tlb_fill`: 具体架构实现 (RISC-V) 处理缺页 (调用 `get_physical_address`, 并在成功后回到 `tlb_set_page`), 在 `/root/qemu-4.1.1/target/riscv/cpu_helper.c:435-475`,
4. `tlb_set_page`: 安装条目, 位于 `cputlb.c:843-849`

我看了这几个函数的实现, 比较有意思的是 `tlb_fill` 这个函数

```
CPUClass *cc = CPU_GET_CLASS(cpu);
bool ok;

/*
 * This is not a probe, so only valid return is success; failure
 * should result in exception + longjmp to the cpu loop.
 */
ok = cc->tlb_fill(cpu, addr, size, access_type, mmu_idx, false, retaddr);
assert(ok);
```

调用 `ok = cc->tlb_fill(...)`, 不直接调用某个架构函数, 而是通过类回调调用了 `riscv_cpu_tlb_fill`。这样可以跳转到不同架构的 `tlb_fill` 函数。我们打入下面几个断点

```
# 1) 指令取指的 TLB 查找入口: get_page_addr_code
(gdb) b get_page_addr_code if (addr == 0xffffffffc02000d6)
# 进入后, 在关键行停下并打印本地变量
(gdb) tbreak /root/qemu-4.1.1/accel/tcg/cputlb.c:1030
(gdb) commands
> silent
> printf "TLB-LOOKUP: mmu_idx=%lu\n", (unsigned long)mmu_idx
> printf "INDEX=%lu\n", (unsigned long)index
> printf "ENTRY.addr_code=%#lx addend=%#lx\n", entry->addr_code, entry->addend
> continue
> end

# 2) 若未命中主表与 victim, 则触发缺页分派 tlb_fill (通用入口)
(gdb) b /root/qemu-4.1.1/accel/tcg/cputlb.c:868 if (addr == 0xffffffffc02000d6)
(gdb) commands
> silent
> printf "TLB-FILL: addr=%#lx access_type=%d mmu_idx=%d\n", addr, access_type, mmu_idx
> continue
> end
```

```
# 3) 架构回调: RISC-V 缺页入口 riscv_cpu_tlb_fill (进入页表翻译)
(gdb) b /root/qemu-4.1.1/target/riscv/cpu_helper.c:435 if (address == 0xffffffffc02000d6)
(gdb) commands
> silent
> printf "RISCV tlb_fill: address=%#lx access_type=%d

mmu_idx=%d\n", address, access_type, mmu_idx
> continue
> end

# 4) 页表翻译核心 (只看结果, 不展开层级调试): get_physical_address 返回点
(gdb) tbreak /root/qemu-4.1.1/target/riscv/cpu_helper.c:459
(gdb) commands
> silent
> printf "TRANSLATE: ret=%d pa=%#lx prot=%d\n", ret, pa, prot
> continue
> end

# 5) 安装软TLB条目 (页对齐), 用于验证 “缺页→安装→再回查命中”
(gdb) tbreak /root/qemu-4.1.1/target/riscv/cpu_helper.c:472
if ((address & ~0xfff) == 0xffffffffc0200000)
(gdb) commands
> silent
> printf "TLB-SET: vaddr=%#lx paddr=%#lx prot=%d

mmu_idx=%d\n", address & ~0xfff, pa & ~0xfff, prot, mmu_idx
> continue
> end

# 6) 回到查找现场, 确认 “安装后命中”: get_page_addr_code 断言命中之后打印 addend
(gdb) tbreak /root/qemu-4.1.1/accel/tcg/cputlb.c:1036
(gdb) commands
> silent
> printf "POST-FILL HIT: entry.addr_code=%#lx addend=%#lx\n", entry->addr_code, entry->addend
> continue
> end
```

输出如下


```
(gdb) c
Continuing.
[Switching to Thread 0x7df29137d640 (LWP 96626)]
TLB-LOOKUP: mmu_idx=3
INDEX=1
ENTRY.addr_code=0xffffffffffffffff addend=0xffffffffffffffff
TRANSLATE: ret=0 pa=0x1000 prot=7
POST-FILL HIT: entry.addr_code=0x1000 addend=0x7df2907ff000

Thread 3 "qemu-system-ris" hit Breakpoint 1, get_page_addr_code (env=0x6479a9dd4910,
addr=18446744072637907158) at /root/qemu-4.1.1/accel/tcg/cputlb.c:1025
1025      uintptr_t mmu_idx = cpu_mmu_index(env, true);
(gdb) c
Continuing.
TLB-FILL: addr=0xffffffffc02000d6 access_type=2 mmu_idx=1
RISCV tlb_fill: address=0xffffffffc02000d6 access_type=2 mmu_idx=1
TLB-SET: vaddr=0xffffffffc0200000 paddr=0x80200000 prot=7 mmu_idx=1
```

前半段（M 模式直映，地址 0x1000）

1. TLB-LOOKUP: mmu_idx=3，表示当前是 RISC-V 的 M 模式
2. INDEX=1，表示该虚拟页在当前 TLB 组中的槽位索引是 1。
3. ENTRY.addr_code=0xffffffffffffffff addend=0xffffffffffffffff。两个 -1 是“未安装/不可用”的哨兵值，说明该页在指令路径上没有有效的 TLB 条目，需要走缺页。
4. TRANSLATE: ret=0 pa=0x1000 prot=7。架构缺页回调完成翻译：M 模式是直映（MBare），虚拟地址直接作为物理地址；ret=0 表示成功，prot=7（R/W/X 全权限）。
5. POST-FILL HIT: entry.addr_code=0x1000 addend=0x7df2907ff000。安装软 TLB 后回到查找现场，确认命中：addr_code 变为页基地址 0x1000；addend 是“主机页基地址 - 虚拟页基地址”的偏移，用于把客体 VA 快速映射到主机 RAM 指针。

后半段（S 模式，高半区页，触发缺页并安装）

1. TLB-FILL: addr=0xffffffffc02000d6 access_type=2 mmu_idx=1。这个过程发生的是数据写访问（access_type=2，来自 kern_init 的 memset），且特权为 S 模式（mmu_idx=1）。因为该页在数据写路径上未命中 TLB，进入通用缺页分派。
 2. RISCV tlb_fill: address=0xffffffffc02000d6 access_type=2 mmu_idx=1。架构缺页入口，进入 RISC-V 的页表翻译（SV39）。
 3. TLB-SET: vaddr=0xffffffffc0200000 paddr=0x80200000 prot=7 mmu_idx=1。安装软 TLB 条目：将虚拟页基 0xffffffffc0200000 映射到物理页基 0x80200000，权限为 prot=7，特权索引 S 模式。之后同页的指令取值与数据访问都会直接命中 TLB。
4. qemu 中模拟出来的 tlb 和我们真实 cpu 中的 tlb 有什么逻辑上的区别

我在上一个部分已经完成了，结论如下：

1. 未分页(M 模式直映):mmu_idx=3,ENTRY.addr_code/addend=-1 → tlb_fill → TRANSLATE: pa=0x1000 prot=7 → POST-FILL HIT
 - (a) 真实 CPU：不走 TLB，直接以物理地址访问；硬件不会“安装” TLB 条目来优化直映路径。

- (b) QEMU: 即使未分页也会安装“软件 TLB”条目, 加速后续访问 (tlb_set_page), 并缓存“主机页指针偏移” addend 以快速形成主机地址。
2. 开启分页(S 模式 SV39): get_page_addr_code → TLB-FILL(access_type=2, mmu_idx=1) → RISCv tlb_fill → TLB-SET vaddr=..., paddr=0x80200000 prot=7
- (a) 真实 CPU: TLB 未命中由硬件页表遍历器走 SV39, 硬件设置 A/D 位, 成功后在硬件 TLB 中缓存 VA→PA 与属性。
- (b) QEMU: TLB 未命中由软件函数链路完成: tlb_fill → riscv_cpu_tlb_fill → get_physical_address → tlb_set_page 安装软 TLB。

(六) gdb 调试系统调用以及返回

1. ecall 调试过程

首先和 lab2 的调试过程一样, 需要采用三终端双 GDB 方法。其中终端 1 启动 QEMU 模拟器, 终端 2 使用 gdb 调试 qemu, 终端 3 使用 gdb 调试 ucore 内核代码。不过与 lab2 不同的是 lab5 需要运行用户态程序。但是 gdb 只加载了内核信息, 所以断点设置会失败。执行如下命令之后就可以加载用户态程序的符号表加载到内核 GDB 中。接着在 syscall.c 文件的第 19 行设置断点, 这其实是 syscall 函数中内联汇编函数的起始地址。

```

1 (gdb) add-symbol-file obj/__user_exit.out
2 add symbol table from file "obj/__user_exit.out"
3 (y or n) y
4 Reading symbols from obj/__user_exit.out...
5 (gdb) break user/libs/syscall.c:18
6 Breakpoint 1 at 0x8000f8: file user/libs/syscall.c, line 19.
7 (gdb) c
8 Continuing.

```

接着在终端 3 输入命令 x/8i \$pc 之后观察到如图1所示结果, 这 8 条汇编指令与 syscall 函数内的前 8 条内联汇编指令一致。接着就使用命令 si 单步执行到 ecall 指令之前。这时为了观察 qemu 如何处理 ecall 指令的, 需要在终端 2 设置断点。

```

(gdb) break user/libs/syscall.c:18
Breakpoint 1 at 0x8000f8: file user/libs/syscall.c, line 19.
(gdb) c
Continuing.

Breakpoint 1, syscall (num=num@entry=30)
  at user/libs/syscall.c:19
19      asm volatile (
(gdb) x/8i $pc
=> 0x8000f8 <syscall+32>:    ld      a0,8(sp)
0x8000fa <syscall+34>:    ld      a1,40(sp)
0x8000fc <syscall+36>:    ld      a2,48(sp)
0x8000fe <syscall+38>:    ld      a3,56(sp)
0x800100 <syscall+40>:    ld      a4,64(sp)
0x800102 <syscall+42>:    ld      a5,72(sp)
0x800104 <syscall+44>:    ecall
0x800108 <syscall+48>:    sd      a0,28(sp)
(gdb)

```

图 1: ecall 断点之后的汇编指令

这个断点必须在处理 `ecall` 指令的那个函数开头，但是我不知道这个函数名也没办法设置断点。这时候就得请大模型出场了。大模型给出的答案如图2所示，它提示我在 `qemu` 中使用图中给出的命令匹配。结果匹配到函数 `trans_ecall`。而且还有函数位置信息，所以使用 `b` 文件路径:行号的命令格式给这个函数设置断点。



图 2: qemu

图3展示出已经成功在 `trans_ecall` 函数开始设置好断点，并在终端 3 继续执行后终端 2 就会触发这个断点。

```
(gdb) break target/riscv/insn_trans/trans_privileged.inc.c:21
Breakpoint 1 at 0x5ea46ca40a2a: file /home/kzhj/公共/riscv/qemu/target/riscv/insn_trans/trans_privileged.inc.c, line 24.
(gdb) c
Continuing.
[Switching to Thread 0x7bcfcc9fe6c0 (LWP 116341)]

Thread 2 "qemu-system-ris" hit Breakpoint 1, trans_ecall (
  ctx=0x7bcfcc9fd7d0, a=0x7bcfcc9fd6d0)
  at /home/kzhj/公共/riscv/qemu/target/riscv/insn_trans/trans_privileged.inc.c:24
24      generate_exception(ctx, RISCV_EXCP_U_ECALL);
(gdb) □
```

图 3: 设置 `trans_ecall` 断点

在终端 2 输入 `list+ 函数名` 命令就可以观察到函数的源码。`trans_ecall` 将硬件层面的 `ecall` 指令, 转化为 `qemu` 能执行的 TCG 中间代码。观察到 `trans_ecall` 函数把 `ecall` 翻译为生成异常, 翻译阶段恒定用 `U-level` 常量, 实际的 `cause` 会在运行时的异常处理。函数内部调用 `generate_exception` 处理用户态 `ecall`。说明该函数将会是接下来的分析对象。

有趣的是后面还调用了函数 `exit_tb`, 注释显示 `no chaining`。于是我就问大模型这个函数调用的作用, `ai` 解释 `qemu` 会将连续的指令序列翻译为一个 TB(翻译块), 后续执行时直接调用 TB 的 TCG 代码, 大幅提升执行效率, `qemu` 会预判下一个 TB 并提前翻译, 形成链状结构。但是在这里 `ecall` 指令执行后会触发异常, 跳转到内核的异常入口, 当前 TB 的后续指令不会被执行, 所以调用这个函数。`ecall` 指令执行后不会返回当前指令流, 所以跳转标记也设置为无返回。

```
1 static bool trans_ecall(DisasContext *ctx, arg_ecall *a)
2 {
3     /* always generates U-level ECALL, fixed in do_interrupt handler */
4     generate_exception(ctx, RISCV_EXCP_U_ECALL);
5     exit_tb(ctx); /* no chaining */
6     ctx->base.is_jmp = DISAS_NORETURN;
```

```

7     return true;
8 }

```

接着给 `generate_exception` 设置断点, 和上一个函数一样找到该函数的位置并设断点。输入命令 `c` 后会触发这个断点。通过 TCG 指令生成, 将异常类型 `excp` 和返回地址 `pc_next` 传递给执行阶段的辅助函数 `gen_helper_raise_exception`。所以接下来要关注的目标是这个函数, 为其打上断点。

```

1 static void generate_exception(DisasContext *ctx, int excp)
2 {
3     tcg_gen_movi_tl(cpu_pc, ctx->base.pc_next);
4     TCGv_i32 helper_tmp = tcg_const_i32(excp);
5     gen_helper_raise_exception(cpu_env, helper_tmp);
6     tcg_temp_free_i32(helper_tmp);
7     ctx->base.is_jump = DISAS_NORETURN;
8 }

```

但是我发现给函数 `gen_helper_raise_exception` 打上断点之后竟然跳转到宏定义 `DEF_HELPER_2` 这里了。带着疑问询问大模型, 它给出的答案如下图所示, `helper` 是通过宏声明的, 调试信息把执行位置映射到了宏的声明处, 而不是我期望的 `gen_helper_raise_exception` 名字上。



图 4: 大模型解惑

接着就使用 `grep` 命令在 `qemu` 目录下找到关于 `raise_exception` 的函数, 看到 `helper_raise_exception` 和 `riscv_raise_exception`。尝试给 `helper_raise_exception` 设置断点并运行, 结果就停在了 `riscv_raise_exception` 函数这里。代码里可以看到 `helper_raise_exception` 调用了 `riscv_raise_exception`。

主要逻辑在后者, 首先会调用 `env_cpu`, 它的作用是将 RISC-V 架构特有的 `CPURISCVState` 结构体转换为 QEMU 通用的 `CPUState` 结构体指针。在 `qemu` 源代码中可以观察到像 `mips`, `riscv` 等架构, `qemu` 的这种做法体现了它的通用性, 实现架构解耦。然后将异常类型 `exception` 通过 `cs->exception_index` 进行存储。后面的 `cpu_loop_exit_restore` 是接下来的调试对象。

```

1 void QEMU_NORETURN riscv_raise_exception(CPURISCVState *env,
2                                           uint32_t exception, uintptr_t pc)
3 {
4     CPUState *cs = env_cpu(env);
5     qemu_log_mask(CPU_LOG_INT, "%s: %d\n", __func__, exception);
6     cs->exception_index = exception;

```

```

7     cpu_loop_exit_restore(cs, pc);
8 }
9 void helper_raise_exception(CPURISCVState *env, uint32_t exception)
10 {
11     riscv_raise_exception(env, exception, 0);
12 }

```

其实之后就是一系列跳转,这里只展示关键步骤,接着调试过程中跳转到 `cpu_handle_exception`, 我们知道在前面传入的异常类型为 `RISCV_EXCP_U_ECALL`, 而且 `riscv_raise_exception` 将 `cpu->exception_index` 设置为 `RISCV_EXCP_U_ECALL`。所以 `ecall` 指令在这个函数里会执行 `else` 分支语句。在这里会直接 `cc->do_interrupt(cpu)`, 大模型解释说在 RISC-V 架构中, CPU-Class 的 `do_interrupt` 成员被初始化为 `riscv_cpu_do_interrupt`。所以接下来应该为这个函数设置断点, 观察它是怎么处理 `ecall` 的。

```

1 static inline bool cpu_handle_exception(CPUState *cpu, int *ret)
2 {
3     // ...
4 #else
5     if (replay_exception()) {
6         CPUClass *cc = CPU_GET_CLASS(cpu);
7         qemu_mutex_lock_iothread();
8         cc->do_interrupt(cpu);
9         qemu_mutex_unlock_iothread();
10        cpu->exception_index = -1;
11    } else if (!replay_has_interrupt()) {
12        /* give a chance to iothread in replay mode */
13        *ret = EXCP_INTERRUPT;
14        return true;
15    }
16 #endif
17 }
18
19 return false;
20 }

```

设置好 `riscv_cpu_do_interrupt` 的断点之后运行, 就会到这个函数开始的地方。可以观察到代码篇幅比较长如果使用 `si` 命令单步执行汇编指令肯定会很困难。所以我就问大模型有没有可以单步执行源码的指令。大模型的回答如图5所示, 使用命令 `n` 就可以单步源码执行, 而且它还解释 `ni` 是汇编层面的单步执行指令, 和 `si` 比较相似。其实它还指出 `p+` 变量的格式指令可以查看变量的值。

```

1 void riscv_cpu_do_interrupt(CPUState *cs)
2 {
3 #if !defined(CONFIG_USER_ONLY)
4
5     RISCVCPU *cpu = RISCV_CPU(cs);
6     CPURISCVState *env = &cpu->env;
7
8     /* cs->exception is 32-bits wide unlike mcause which is XLEN-bits wide

```

```

9      * so we mask off the MSB and separate into trap type and cause.
10     */
11     bool async = !(cs->exception_index & RISCV_EXCP_INT_FLAG);
12     target_ulong cause = cs->exception_index & RISCV_EXCP_INT_MASK;
13     target_ulong deleg = async ? env->mideleg : env->medeleg;
14     target_ulong tval = 0;
15     //...篇幅较长此处省略一部分
16     if (env->priv <= PRV_S &&
17         cause < TARGET_LONG_BITS && ((deleg >> cause) & 1)) {
18         /* handle the trap in S-mode */
19         target_ulong s = env->mstatus;
20         s = set_field(s, MSTATUS_SPIE, env->priv_ver >= PRIV_VERSION_1_10_0 ?
21             get_field(s, MSTATUS_SIE) : get_field(s, MSTATUS_UIE << env->priv
22                 ));
23         s = set_field(s, MSTATUS_SPP, env->priv);
24         s = set_field(s, MSTATUS_SIE, 0);
25         env->mstatus = s;
26         env->scause = cause | ((target_ulong)async << (TARGET_LONG_BITS - 1))
27             ;
28         env->sepc = env->pc;
29         env->sbadaddr = tval;
30         env->pc = (env->stvec >> 2 << 2) +
31             ((async && (env->stvec & 3) == 1) ? cause * 4 : 0);
32         riscv_cpu_set_mode(env, PRV_S);
33     } else {
34         /* handle the trap in M-mode */
35         target_ulong s = env->mstatus;
36         s = set_field(s, MSTATUS_MPIE, env->priv_ver >= PRIV_VERSION_1_10_0 ?
37             get_field(s, MSTATUS_MIE) : get_field(s, MSTATUS_UIE << env->priv
38                 ));
39         s = set_field(s, MSTATUS_MPP, env->priv);
40         s = set_field(s, MSTATUS_MIE, 0);
41         env->mstatus = s;
42         env->mcause = cause | ~(((target_ulong)-1) >> async);
43         env->mepc = env->pc;
44         env->mbadaddr = tval;
45         env->pc = (env->mtvec >> 2 << 2) +
46             ((async && (env->mtvec & 3) == 1) ? cause * 4 : 0);
47         riscv_cpu_set_mode(env, PRV_M);
48     }
49
50     /* NOTE: it is not necessary to yield load reservations here. It is only
51     * necessary for an SC from "another hart" to cause a load reservation
52     * to be yielded. Refer to the memory consistency model section of the
53     * RISC-V ISA Specification.
54     */
55 #endif

```



```

54     cs->exception_index = EXCP_NONE; /* mark handled to qemu */
55 }

```

三、解决方法：如何快速跳过这行，进入下一行源码？

如果你不想逐指令执行，想要按「源码行」单步执行（即一次 `ni` 走完一条源码行的所有指令），可以使用以下方法：

方法 1：使用 `n` 命令（推荐，源码模式下的“下一行”）

`n` 是 GDB 的 `next` 命令，核心是按源码行单步执行，会自动执行完当前源码行对应的所有机器指令，直接进入下一行源码：

```

gdb ^
(gdb) n # 执行后直接跳到下一行: CPURISCVState *env = &cpu->env;

```

注意：`n` 和 `ni` 的区别（再次强调）：

命令	执行单位	行为特点	适用场景
<code>n</code>	源码行	一次走完当前行的所有指令	源码调试，聚焦业务逻辑
<code>ni</code>	机器指令	一次执行一条指令	汇编调试，分析底层实现

图 5：单条源码指令执行方法

所以接下来需要看看这个函数如何处理 `ecall` 的。在这个函数里使用命令 `n` 和 `p` 进行调试。首先初始化异常信息 `async`，由于 `ecall` 是同步异常使用 `async` 为 `false`，这也用 `p async` 进行验证。接着还会初始化 `cause`，`medeleg`，`tval`。同步异常使用 `medeleg` 寄存器，用于判断是否将异常委派给 S-mode 处理。

之后 `ecall` 会进入 `if (!async)` 分支，`ecall` 无错误地址，走 `default` 分支，直接 `break`。这里是用户态 `ecall` 所以在 `ecall_cause_map` 里根据当前特权级映射 `ecall` 原因。而在判断进入哪个模式时它进入了 `s` 态。之后实现了下面 4 个方面的修改。之后就进入内核态处理 `ecall` 异常。

修改 `mstatus` 寄存器：保存中断状态，原特权级，关闭当前模式中断。

保存异常上下文：将 `ecall` 的原因、发生异常的 PC、错误地址分别存入 `scause`，`epc`，`mbadaddr`。

设置内核入口地址：将 `env->pc` 指向 `mtvec`，这是 M-mode 陷阱向量基址，内核初始化的异常入口地址，实现从用户态到内核态的地址跳转。

切换特权级：最后将 CPU 特权级从 `PRV_U` 切换到 `PRV_M`。

2. sret 调试过程

接着分析 `qemu` 如何处理 `sret` 指令的。大模型找到最开始处理 `sret` 的函数是 `trans_sret`，将 `riscv` 的 `sret` 指令翻译为 TCG 中间代码。`CONFIG_USER_ONLY` 这个宏定义表示仅模拟用户态程序。接着检查 CPU 是否支持 `s` 态，若不支持则翻译失败；若支持则生成 TCG 代码调用 `gen_helper_sret` 辅助函数执行指令逻辑，随后终止当前翻译块、禁止链化，并标记翻译块为无返回，最终返回成功标志完成 `sret` 指令。这个过程和 `ecall` 指令翻译过程大致相同。

看到函数里调用 `gen_helper_sret`，为其设置断点，运行之后发现也是停在了 `DEF_HELPER_2`，不过它的参数是 `sret`。前面是 `raise_exception`。

```

1 static bool trans_sret(DisasContext *ctx, arg_sret *a)
2 {
3 #ifndef CONFIG_USER_ONLY
4     tcg_gen_movi_tl(cpu_pc, ctx->base.pc_next);
5
6     if (has_ext(ctx, RVS)) {
7         gen_helper_sret(cpu_pc, cpu_env, cpu_pc);
8         exit_tb(ctx); /* no chaining */
9         ctx->base.is_jmp = DISAS_NORETURN;
10    } else {
11        return false;
12    }
13    return true;
14 #else
15    return false;
16 #endif
17 }

```

之后询问大模型接下来调用的函数是 helper_sret。sret 是 riscv 的特权返回指令，仅能在 S 态或 M 态执行，作用是从 S 态的陷阱（比如 ecall）中返回，恢复到之前的特权级，并更新状态寄存器。这个函数就是对该硬件语义的软件模拟。主要代码如下所示，单步 n 命令调试执行源码。

```

1 target_ulong helper_sret(CPURISCVState *env, target_ulong cpu_pc_deb)
2 {
3     if (!(env->priv >= PRV_S)) {
4         riscv_raise_exception(env, RISCV_EXCP_ILLEGAL_INST, GETPC());
5     }
6     target_ulong retpc = env->sepc;
7     if (!riscv_has_ext(env, RVC) && (retpc & 0x3)) {
8         riscv_raise_exception(env, RISCV_EXCP_INST_ADDR_MIS, GETPC());
9     }
10    if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
11        get_field(env->mstatus, MSTATUS_TSR)) {
12        riscv_raise_exception(env, RISCV_EXCP_ILLEGAL_INST, GETPC());
13    }
14    target_ulong mstatus = env->mstatus;
15    target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP);
16    mstatus = set_field(mstatus,
17        env->priv_ver >= PRIV_VERSION_1_10_0 ?
18        MSTATUS_SIE : MSTATUS_UIE << prev_priv,
19        get_field(mstatus, MSTATUS_SPIE));
20    mstatus = set_field(mstatus, MSTATUS_SPIE, 0);
21    mstatus = set_field(mstatus, MSTATUS_SPP, PRV_U);
22    riscv_cpu_set_mode(env, prev_priv);
23    env->mstatus = mstatus;
24    return retpc;
25 }

```


如图6所示，单步源码调试之后没有触发任何异常，说明 sret 指令处理成功。最后完成状态更新与特权级切换。并返回 retpc。TCG 把返回值写入临时 cpu_pc，随后执行 exit_tb，调用 TB epilogue，完成寄存器状态写回并返回到 tcg_cpu_exec，然后 CPU 线程回到 qemu 调度主循环，继续从 retpc 恢复的特权级用户态执行。随后我使用 delete 删除终端 2 和终端 3 的断点。如图7所示终端 1 就可以正常执行完后面的过程。到此系统调用以及返回的调试过程结束。

```

76         if (!(env->priv >= PRIV_S)) {
(gdb) n
80         target_ulong retpc = env->sepc;
(gdb) n
81         if (!riscv_has_ext(env, RVC) && (retpc & 0x3)) {
(gdb) n
85         if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
(gdb) n
86             get_field(env->mstatus, MSTATUS_TSR)) {
(gdb) n
85         if (env->priv_ver >= PRIV_VERSION_1_10_0 &&
(gdb) n
90             target_ulong mstatus = env->mstatus;
(gdb) n
91             target_ulong prev_priv = get_field(mstatus, MSTATUS_SPP);
(gdb) n
92             mstatus = set_field(mstatus,
(gdb) n
96             mstatus = set_field(mstatus, MSTATUS_SPIE, 0);
(gdb) n
97             mstatus = set_field(mstatus, MSTATUS_SPP, PRIV_U);
(gdb) n
98             riscv_cpu_set_mode(env, prev_priv);
(gdb) n
99             env->mstatus = mstatus;
-

```

图 6: sret 核心处理过程

```

kzhj@kzhj-VMware-Virtual-Platform:~/公共/labcode/lab5
$ make debug
end 0xc02aa784 (virtual)
Kernel executable memory footprint: 682KB
memory management: default_pmm_manager
physcial memory map:
memory: 0x08000000, [0x80000000, 0x87ffffff].
vapaofset is 18446744070488326144
check_alloc_page() succeeded!
check_pgdir() succeeded!
check_boot_pgdir() succeeded!
use SLOB allocator
kmmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
++ setup timer interrupts
kernel_execve: pid = 2, name = "exit".
Breakpoint
I am the parent. Forking the child...
I am parent, fork a child pid 3
I am the parent, waiting now..
I am the child.
waitpid 3 ok.
exit pass.
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:515:
initproc exit.

```

图 7: 调试结束

四、 重要知识点总结

(一) 本实验中的重要知识点

本实验中最重要知识点之一为用户态与内核态的区分与特权级隔离机制。在操作系统原理中，通常会讲到用户态程序不能直接访问硬件资源，而内核态拥有更高的执行权限，这样的设计可以防止用户程序破坏系统的稳定性。在本实验中，这一原理通过 RISC-V 的 U 模式和 S 模式得到了具体体现。用户程序只能在 U 模式下运行，任何需要操作系统支持的行为都必须通过系统调用进入 S 模式完成。相比课堂上的理论描述，实验中的实现让这一机制变得更加直观，能够从代码和寄存器设置中清楚地看到特权级切换是如何发生的。

另一个非常关键的知识点是系统调用的实现机制。在操作系统原理中，系统调用通常被描述为用户程序与操作系统之间的接口，但对其内部实现过程往往只进行概括性说明。在本实验中，通过分析 `syscall`、`trap` 以及相关汇编代码，可以清晰地看到系统调用从用户态触发、进入异常处理、在内核态完成具体服务、再返回用户态的完整流程。实验使我认识到，系统调用不仅是一个函数调用，更是依赖硬件异常机制完成的一次特权级转换过程。

本实验还加深了我对进程创建与执行模型（`fork` / `exec`）的理解。在操作系统原理中，`fork` 和 `exec` 通常被作为 Unix 进程模型的核心思想进行介绍，即通过 `fork` 复制进程，通过 `exec` 加载新程序。在实验中，`fork` 通过复制进程控制块和地址空间创建子进程，而 `exec` 则释放原有的用户空间并重新加载 ELF 程序。实验实现与理论模型在整体思想上是一致的，只是在实现细节上进行了简化，使得 `fork` 和 `exec` 的作用更加清晰，也更容易理解它们各自承担的职责。

此外，用户进程的地址空间管理也是本实验中的一个重要知识点。操作系统原理中强调每个进程拥有独立的虚拟地址空间，而在实验中，这一概念通过页表复制和内存映射得到了具体体现。通过 `copy_range` 函数复制父进程的用户空间，可以直观地理解“进程之间内存相互隔离”这一抽象概念。同时，在实验中引入的 Copy-on-Write 机制设计讨论，也让我认识到操作系统在保证正确性的同时，还需要考虑性能和资源利用效率。

最后，本实验还涉及了进程状态与生命周期管理。在操作系统原理中，进程通常会经历创建、就绪、运行、阻塞和退出等状态。在实验中，这些状态通过 `proc_struct` 中的状态字段以及 `wait`、`exit` 等系统调用被具体实现出来。通过阅读代码和分析执行流程，可以清楚地看到进程状态是如何随着系统调用和调度过程发生变化的，这比单纯记忆状态转换图更加直观和易于理解。

(二) 实验中较少强调的知识点

在本实验中，还有一些内容在操作系统原理课程中并未被重点展开，但在实际实现中却十分关键。例如，系统第一次从内核态进入用户态的过程在理论课程中往往被简化为“启动第一个用户进程”，而在实验中则需要通过构造 `trapframe` 并借助异常返回机制来完成。这一过程让我意识到，很多在理论中看似简单的步骤，在真实系统中都需要复杂而严谨的实现支持。

此外，实验中还体现了异常处理机制的复用思想。`ucore` 并没有为用户态启动单独设计一套切换机制，而是复用了系统调用和异常返回路径。这种设计虽然在课程理论中很少强调，但在实际系统中非常常见，体现了操作系统实现中对代码复用和机制统一的重视。

(三) 本实验中没有实现的知识点

尽管本实验覆盖了进程管理和系统调用的核心内容，但仍有一些操作系统原理中的重要知识点在实验中没有得到体现。首先是文件系统相关机制。在真实操作系统中，用户程序通常通过文件系统从磁盘加载，而本实验为了简化实现，将用户程序直接链接进内核镜像中，从而未涉及文件系统、磁盘 I/O 以及权限检查等内容。

其次，调度算法的具体策略在实验中也未深入体现。实验中只关注进程是否能够被正确创建和切换，而操作系统原理中讲到的优先级调度、公平性以及多级反馈队列等调度算法并未展开实现。

另外，进程间通信与同步机制也是操作系统原理中的重要组成部分，例如信号、管道和信号量等，但在本实验中尚未涉及。这些机制通常需要在更复杂的内核环境下才能实现。

最后，系统安全与权限管理机制在操作系统原理中同样十分重要，例如用户身份、权限控制等内容，但在本实验中未体现。这也是由于 Lab5 的重点在于用户进程与系统调用的基础实现，而非完整操作系统功能。

五、 实验总结

通过本次 Lab5 实验，我对用户程序在操作系统中是如何被创建、加载并运行起来有了更加完整和直观的认识。相比之前实验中只涉及内核线程的情况，本实验首次引入了用户态进程，使我能够从整体上理解操作系统如何在保证安全性的前提下运行普通用户程序。通过实际阅读和修改 ucore 源码，我逐步将课堂上抽象的“用户态与内核态”“系统调用”等概念与具体实现过程联系起来，对操作系统的运行机制有了更加清晰的认识。

在实验过程中，我对系统调用的实现流程理解得尤为深入。以前在学习操作系统原理时，系统调用更多被当作一个逻辑接口来理解，而通过本实验可以清楚地看到，从用户程序发起系统调用，到 `ecall` 指令触发异常，再到内核中完成处理并通过 `sret` 返回用户态，这一过程离不开硬件特权级机制与内核异常处理代码的配合。这使我认识到，系统调用并不仅仅是函数层面的封装，而是操作系统与硬件共同实现的一套完整机制。

本实验还加深了我对进程管理模型的理解，尤其是 `fork` 和 `exec` 的分工设计。通过分析和实现相关代码，我逐渐理解了为什么 `fork` 负责创建进程执行环境，而 `exec` 负责加载新的程序映像，这种设计使进程创建更加灵活，也更符合操作系统的整体结构。同时，通过 `wait` 和 `exit` 的实现过程，可以看到操作系统是如何在进程退出后回收资源，并保证父子进程之间的同步关系，从而避免资源泄露和系统混乱。

此外，本实验让我对虚拟内存和地址空间隔离有了更直观的认识。在 `fork` 过程中对子进程地址空间的复制，使我能够从实现角度理解“每个进程拥有独立虚拟地址空间”这一理论结论。对 Copy-on-Write 机制的设计分析，也让我意识到操作系统在实际实现中不仅要保证功能正确，还需要在性能和资源利用之间进行权衡。

总体来看，Lab5 是一次将操作系统原理与具体实现紧密结合的实验。通过本实验，我不仅掌握了用户进程和系统调用的基本实现方式，也逐渐培养了从源码角度理解操作系统的能力。这为后续学习更复杂的操作系统功能（如文件系统、进程调度和内存优化机制）打下了良好的基础，也让我对操作系统这门课程的整体结构和工程复杂性有了更加深入的认识。